

HƯỚNG DẪN THỰC HÀNH ORACLE SQL/PL/SQL - BUỔI 4

View · Trigger · Stored Procedure · Bài Tập Tổng Hợp

**Schema: Quản lý Khóa Học - COURSE, CLASS, STUDENT,
ENROLLMENT, INSTRUCTOR, GRADE**

Phần 1: Hệ thống kiến thức | Phần 2+3+4+5: Hướng dẫn từng câu

PHẦN 1: HỆ THỐNG KIẾN THỨC BUỔI 4

1.1. View - Bảng Ảo Trong Oracle

View là một câu truy vấn SELECT được đặt tên và lưu trong database. View không lưu dữ liệu thực - mỗi khi truy vấn view, Oracle thực thi câu SELECT bên dưới và trả kết quả.

Cú pháp tạo và quản lý View

```
-- Tạo view mới
CREATE [OR REPLACE] VIEW ten_view
[(alias_cot1, alias_cot2, ...)]
AS
SELECT ... FROM ... WHERE ...;

-- Xóa view
DROP VIEW ten_view;

-- Xem định nghĩa view
SELECT text FROM user_views WHERE view_name = 'TEN_VIEW';

-- Kiểm tra trạng thái view
SELECT object_name, status FROM user_objects WHERE object_type = 'VIEW';
```

Các tùy chọn quan trọng khi tạo View

Tùy chọn	Ý nghĩa	Ví dụ / Ghi chú
OR REPLACE	Tạo mới hoặc ghi đè view đã tồn tại - không cần DROP trước	CREATE OR REPLACE VIEW v AS ...
WITH READ ONLY	Ngăn mọi thao tác INSERT/UPDATE/DELETE qua view	CREATE VIEW v AS SELECT... WITH READ ONLY;
WITH CHECK OPTION	Dữ liệu INSERT/UPDATE qua view phải thỏa mãn điều kiện WHERE của view	CREATE VIEW v AS SELECT... WHERE dept=10 WITH CHECK OPTION;
FORCE	Tạo view dù bảng cơ sở chưa tồn tại (hữu ích khi lập trình sẵn)	CREATE FORCE VIEW v AS ...

View có thể cập nhật (Updatable View) - Điều kiện

Không phải view nào cũng cho phép INSERT/UPDATE/DELETE. View chỉ có thể cập nhật khi:

- Truy vấn chỉ dựa trên 1 bảng cơ sở (hoặc kết hợp các bảng theo cách đơn giản)
- Không có: DISTINCT, GROUP BY, HAVING, UNION, hàm gộp nhóm (SUM, COUNT...)
- Không có: subquery trong WHERE, biểu thức tính toán trong SELECT

Phân biệt WITH READ ONLY và WITH CHECK OPTION

Thuộc tính	INSERT	UPDATE (thỏa WHERE)	UPDATE (vi phạm WHERE)
Không có tùy chọn	Cho phép	Cho phép	Cho phép (hàng biến mất khỏi view)
WITH CHECK OPTION	Nếu thỏa WHERE	Cho phép	Từ chối (ORA-01402)
WITH READ ONLY	Từ chối (ORA-42399)	Từ chối (ORA-42399)	Từ chối (ORA-42399)

Hàm phân tích (Analytic Functions) dùng trong View

```
-- RANK(): xếp hạng, có thể trùng (1,1,3)
SELECT courseno, description,
       RANK() OVER (ORDER BY tong_dk DESC) AS hang
FROM ...

-- DENSE_RANK(): xếp hạng liên tục (1,1,2)
SELECT ..., DENSE_RANK() OVER (ORDER BY salary DESC) AS hang FROM ...

-- ROW_NUMBER(): số thứ tự tuyệt đối, không trùng lặp (1,2,3)
SELECT ..., ROW_NUMBER() OVER (ORDER BY salary DESC) AS stt FROM ...

-- Lấy 5 hàng đầu sau khi xếp hạng (Oracle 12c+)
SELECT * FROM (
  SELECT ..., RANK() OVER (ORDER BY tong_dk DESC) AS hang FROM ...
) WHERE hang <= 5;
```

1.2. Stored Procedure - Kỹ Thuật Nâng Cao

Cấu trúc đầy đủ và các kỹ thuật hay dùng

```
CREATE OR REPLACE PROCEDURE ten_procedure
  (p_in IN kieu,
   p_out OUT kieu)
IS
  -- Khai báo biến cục bộ
  v_count NUMBER;
  v_old   NUMBER;
BEGIN
  -- 1. Kiểm tra điều kiện đầu vào
  SELECT COUNT(*) INTO v_count FROM bang WHERE id = p_in;
  IF v_count = 0 THEN
    DBMS_OUTPUT.PUT_LINE('Không tìm thấy!');
    RETURN; -- Thoát sớm, không làm gì thêm
  END IF;

  -- 2. Lưu giá trị cũ trước khi thay đổi
  SELECT cot INTO v_old FROM bang WHERE id = p_in;

  -- 3. Thực hiện thay đổi
  UPDATE bang SET cot = p_out WHERE id = p_in;
```

```

-- 4. COMMIT sau khi thanh cong
COMMIT;
DBMS_OUTPUT.PUT_LINE('Thanh cong! Cu: ' || v_old || ' -> Moi: ' || p_out);

EXCEPTION
  WHEN OTHERS THEN
    ROLLBACK;
    DBMS_OUTPUT.PUT_LINE('Loi: ' || SQLERRM);
END ten_procedure;
/

```

Kỹ thuật MERGE INTO - INSERT hoặc UPDATE trong một lệnh

MERGE INTO là lệnh mạnh mẽ dùng khi cần 'INSERT nếu chưa có, UPDATE nếu đã có':

```

MERGE INTO bang_dich d
USING (SELECT :gia_tri_khoa AS id_key FROM DUAL) s
ON (d.id = s.id_key)
WHEN MATCHED THEN
  UPDATE SET d.cot = gia_tri_moi
WHEN NOT MATCHED THEN
  INSERT (d.id, d.cot) VALUES (s.id_key, gia_tri_moi);

-- Ví dụ thực tế: đồng bộ điểm từ ENROLLMENT sang GRADE
MERGE INTO grade g
USING (SELECT studentid, classid, finalgrade
      FROM enrollment
      WHERE studentid = p_sid AND classid = p_cid) src
ON (g.studentid = src.studentid AND g.classid = src.classid)
WHEN MATCHED THEN
  UPDATE SET g.grade = src.finalgrade
WHEN NOT MATCHED THEN
  INSERT (g.studentid, g.classid, g.grade, g.createdby, g.createddate,
        g.modifiedby, g.modifieddate)
  VALUES (src.studentid, src.classid, src.finalgrade,
        USER, SYSDATE, USER, SYSDATE);

```

Định dạng output với DBMS_OUTPUT.PUT_LINE và LPAD/RPAD

```
-- Can chỉnh số thu tu (STT) trong báo cáo
DBMS_OUTPUT.PUT_LINE(LPAD(v_stt, 3) || ' | '      -- STT can phải, rộng 3
                        || RPAD(v_ten, 20) || ' | '  -- Ten can trái, rộng 20
                        || LPAD(v_diem, 8) || ' | '  -- Diem can phải, rộng 8
                        || v_xep_loai);

-- In dòng kẻ ngang
DBMS_OUTPUT.PUT_LINE(RPAD('-',50,'-'));

-- In số với định dạng tiền tệ
DBMS_OUTPUT.PUT_LINE('Hoc phi: ' || TO_CHAR(v_cost, '999,990.00'));
```

1.3. Trigger - Kỹ Thuật Nâng Cao**Phân loại Trigger trong Oracle**

Loại	Đặc điểm	Khi nào dùng
BEFORE / FOR EACH ROW	Chạy trước mỗi hàng bị thay đổi. Có thể thay đổi :NEW	Kiểm tra điều kiện, gán giá trị mặc định, validate
AFTER / FOR EACH ROW	Chạy sau mỗi hàng bị thay đổi. :NEW đã được ghi	Ghi audit log, cập nhật bảng khác
BEFORE (mức bảng)	Chạy 1 lần duy nhất, không có :NEW/:OLD	Kiểm tra toàn bộ thao tác (không phụ thuộc từng hàng)
AFTER (mức bảng)	Chạy 1 lần sau khi toàn bộ thao tác hoàn tất	Ghi log tổng hợp, cập nhật thống kê
INSTEAD OF	Thay thế hoàn toàn thao tác DML - dùng cho VIEW	Cho phép INSERT/UPDATE/DELETE qua view phức tạp

Vị ngữ điều kiện trong Trigger - INSERTING / UPDATING / DELETING

```
CREATE OR REPLACE TRIGGER trg_example
  BEFORE INSERT OR UPDATE OR DELETE ON bang
  FOR EACH ROW
BEGIN
  IF INSERTING THEN
```

```
-- Chỉ chạy khi là INSERT
:NEW.created_by := USER;
:NEW.created_date := SYSDATE;
END IF;

IF UPDATING THEN
    -- Chỉ chạy khi là UPDATE
    -- UPDATING('ten_cot'): kiểm tra cột cụ thể có bị update không
    IF UPDATING('finalgrade') THEN
        DBMS_OUTPUT.PUT_LINE('Điểm vừa được cập nhật!');
    END IF;
END IF;

IF DELETING THEN
    -- Chỉ chạy khi là DELETE
    -- :OLD còn dữ liệu, :NEW là NULL
    NULL; -- Không làm gì
END IF;
END;
/
```

RAISE_APPLICATION_ERROR - Tạo lỗi tùy chỉnh

```
-- Cu pháp
RAISE_APPLICATION_ERROR(ma_loi, 'Thông báo lỗi');
-- ma_loi: phải nằm trong khoảng -20000 đến -20999

-- Ví dụ trong trigger:
IF v_so_sv >= v_capacity THEN
    RAISE_APPLICATION_ERROR(
        -20010,
        'Lớp ' || :NEW.classid || ' đã đầy! (' ||
        v_so_sv || '/' || v_capacity || ' chỗ)'
    );
END IF;

-- Bật lỗi tùy chỉnh trong PL/SQL:
BEGIN
    INSERT INTO enrollment ...
EXCEPTION
```

```
WHEN OTHERS THEN
  IF SQLCODE = -20010 THEN
    DBMS_OUTPUT.PUT_LINE('Lop da day!');
  END IF;
END;
```

Phòng tránh lỗi ORA-04091: Table is Mutating

Lỗi này xảy ra khi trigger đọc hoặc ghi chính bảng đang bị thay đổi. Ví dụ: trigger AFTER INSERT trên ENROLLMENT đọc COUNT(*) từ ENROLLMENT - Oracle không cho phép vì bảng đang bị thay đổi.

Cách khắc phục: Dùng BEFORE thay vì AFTER (thường giải quyết được), hoặc dùng Compound Trigger (Oracle 11g+):

```
-- Compound Trigger (Oracle 11g+): chia trigger thành nhiều giai đoạn
CREATE OR REPLACE TRIGGER trg_compound
  FOR INSERT ON enrollment
  COMPOUND TRIGGER

  -- Biến dùng chung giữa các giai đoạn
  v_count NUMBER;

  BEFORE EACH ROW IS
  BEGIN
    -- Chạy trước mỗi hàng INSERT
    SELECT COUNT(*) INTO v_count FROM enrollment WHERE classid =
    :NEW.classid;
    IF v_count >= (SELECT capacity FROM class WHERE classid = :NEW.classid)
    THEN RAISE_APPLICATION_ERROR(-20010, 'Lop day roi!');
    END IF;
  END BEFORE EACH ROW;

END trg_compound;
/
```

1.4. So Sánh Nhanh: View vs Procedure vs Trigger

Tiêu chí	View	Procedure	Trigger
Gọi như thế nào	SELECT * FROM view_name	EXEC proc_name(args) hoặc gọi từ PL/SQL	Tự động khi có sự kiện DML
Trả về giá trị	Tập kết quả (bảng ảo)	Qua tham số OUT hoặc DBMS_OUTPUT	Không trả về - chỉ thực thi
Có tham số	Không	Có (IN / OUT / IN OUT)	Không (nhưng dùng :NEW/:OLD)
Mục đích chính	Đơn giản hóa truy vấn phức tạp, bảo mật cột	Đóng gói logic nghiệp vụ, tái sử dụng code	Tự động hóa, ràng buộc nghiệp vụ, audit
Lưu trong DB	Chỉ lưu định nghĩa (không lưu dữ liệu)	Lưu code đã biên dịch	Lưu code đã biên dịch, gắn với bảng

PHẦN 2: HƯỚNG DẪN BÀI 1 - VIEW

Câu 1.1: Tạo view vw_course_summary - thông tin tổng quan môn học

Phân tích: Kết 3 bảng: COURSE → CLASS → ENROLLMENT. GROUP BY theo môn học. Đếm số lớp và tổng SV.

Phân tích cú pháp JOIN cần dùng:

- COURSE và CLASS kết qua CourseNo
- CLASS và ENROLLMENT kết qua ClassID
- Dùng LEFT JOIN để giữ lại môn học chưa có lớp (COUNT = 0)

Code đầy đủ:

```
CREATE OR REPLACE VIEW vw_course_summary AS
SELECT co.courseno,
       co.description,
       co.cost,
       COUNT(DISTINCT cl.classid) AS so_lop,
       COUNT(e.studentid) AS tong_sv
```



```

FROM course co
  LEFT JOIN class cl ON co.courseno = cl.courseno
  LEFT JOIN enrollment e ON cl.classid = e.classid
GROUP BY co.courseno, co.description, co.cost
ORDER BY tong_sv DESC;

```

-- Kiểm tra view:

```
SELECT * FROM vw_course_summary;
```

Giải thích: Dùng COUNT(DISTINCT cl.classid) để đếm số lớp phân biệt (tránh đếm trùng khi JOIN với ENROLLMENT). COUNT(e.studentid) tự bỏ qua NULL nên đếm đúng số SV đã đăng ký.

Câu 1.2: Tạo view vw_student_status - thông tin sinh viên và tình trạng học tập

Phân tích: Kết 4 bảng: STUDENT → ENROLLMENT → CLASS → COURSE. GROUP BY theo sinh viên. Lọc chỉ SV đang học ít nhất 1 lớp bằng HAVING.

```

CREATE OR REPLACE VIEW vw_student_status AS
SELECT s.studentid,
       s.firstname || ' ' || s.lastname AS ho_ten,
       COUNT(e.classid) AS so_lop_hoc,
       NVL(SUM(co.cost), 0) AS tong_hoc_phi,
       ROUND(AVG(e.finalgrade), 2) AS diem_tb
FROM student s
  JOIN enrollment e ON s.studentid = e.studentid
  JOIN class cl ON e.classid = cl.classid
  JOIN course co ON cl.courseno = co.courseno
GROUP BY s.studentid, s.firstname, s.lastname
HAVING COUNT(e.classid) >= 1
ORDER BY s.studentid;

SELECT * FROM vw_student_status;

```

Lưu ý: NVL(SUM(co.cost), 0) tránh trả về NULL khi sinh viên chưa có học phí. AVG bỏ qua NULL tự động nên diem_tb chỉ tính hàng có điểm.

Câu 1.3: Tạo view vw_class_availability - lớp học còn chỗ trống

Phân tích: Kết COURSE, CLASS, INSTRUCTOR, ENROLLMENT. Tính số chỗ còn trống. CASE WHEN cho cột trạng thái. WHERE lọc chỉ lớp còn chỗ.

```
CREATE OR REPLACE VIEW vw_class_availability AS
SELECT cl.classid,
       cl.courseno,
       co.description,
       i.firstname || ' ' || i.lastname    AS ten_giao_vien,
       cl.capacity,
       COUNT(e.studentid)                 AS so_da_dk,
       cl.capacity - COUNT(e.studentid)    AS cho_trong,
       CASE
         WHEN cl.capacity - COUNT(e.studentid) > 0 THEN 'Con cho'
         ELSE 'Het cho'
       END                                AS trang_thai
FROM   class cl
       JOIN course co ON cl.courseno = co.courseno
       JOIN instructor i ON cl.instructorid = i.instructorid
       LEFT JOIN enrollment e ON cl.classid = e.classid
GROUP BY cl.classid, cl.courseno, co.description,
         i.firstname, i.lastname, cl.capacity
HAVING cl.capacity - COUNT(e.studentid) > 0
ORDER BY cl.classid;
SELECT * FROM vw_class_availability;
```

Giải thích: HAVING thay vì WHERE vì điều kiện lọc (cho_trong > 0) phụ thuộc vào hàm gộp nhóm COUNT. Không thể đặt điều kiện này trong WHERE.

Câu 1.4: Tạo view vw_top_courses - chỉ đọc, top 5 môn được đăng ký nhiều nhất

Phân tích: Dùng hàm phân tích RANK() OVER. Lọc 5 hàng đầu bằng Inline View. Thêm WITH READ ONLY.

```
CREATE OR REPLACE VIEW vw_top_courses AS
SELECT courseno, description, cost, tong_dk, hang
FROM (
  SELECT co.courseno,
         co.description,
         co.cost,
         COUNT(e.studentid)                AS tong_dk,
         RANK() OVER (ORDER BY COUNT(e.studentid) DESC) AS hang
  FROM   course co
        LEFT JOIN class cl ON co.courseno = cl.courseno
        LEFT JOIN enrollment e ON cl.classid = e.classid
  GROUP BY co.courseno, co.description, co.cost
)
WHERE hang <= 5
ORDER BY hang
WITH READ ONLY;

SELECT * FROM vw_top_courses;

-- Thu INSERT vào view này (sẽ báo lỗi ORA-42399):
INSERT INTO vw_top_courses (courseno, description, cost)
VALUES (999, 'Test', 100);

-- Oracle báo: ORA-42399: cannot perform a DML operation on a read-only view
```

Lưu ý: RANK() OVER không thể dùng trực tiếp trong WHERE - phải bọc trong Inline View (subquery trong FROM) để có thể lọc WHERE hang <= 5.

Câu 1.5: Tạo view vw_pending_enrollment với WITH CHECK OPTION và kiểm tra

Phân tích: View chỉ hiển thị ENROLLMENT chưa có điểm. WITH CHECK OPTION đảm bảo INSERT qua view cũng phải thỏa điều kiện WHERE.

```
CREATE OR REPLACE VIEW vw_pending_enrollment AS
SELECT studentid, classid, enrolldate, finalgrade,
       createdby, createddate, modifiedby, modifieddate
FROM   enrollment
WHERE  finalgrade IS NULL
WITH CHECK OPTION;

SELECT * FROM vw_pending_enrollment;

-- INSERT 1: FinalGrade = NULL -> THANH CONG (thỏa điều kiện WHERE)
INSERT INTO vw_pending_enrollment
(studentid, classid, enrolldate, createdby, createddate,
 modifiedby, modifieddate)
VALUES (999, 1, SYSDATE, USER, SYSDATE, USER, SYSDATE);
-- INSERT 2: FinalGrade = 85 -> LOI ORA-01402 (vi phạm WITH CHECK OPTION)
INSERT INTO vw_pending_enrollment
(studentid, classid, enrolldate, finalgrade,
 createdby, createddate, modifiedby, modifieddate)
VALUES (998, 1, SYSDATE, 85, USER, SYSDATE, USER, SYSDATE);
-- ORA-01402: view WITH CHECK OPTION where-clause violation
```

Giải thích: INSERT 2 bị từ chối vì nếu Oracle cho phép INSERT với FinalGrade=85, hàng đó sẽ không thỏa mãn WHERE finalgrade IS NULL - tức là hàng vừa thêm vào sẽ không hiển thị trong view. WITH CHECK OPTION ngăn điều này xảy ra.

PHẦN 3: HƯỚNG DẪN BÀI 2 - STORED PROCEDURE**Câu 2.1:** Thủ tục enroll_student - đăng ký sinh viên vào lớp học

Phân tích: Kiểm tra 5 điều kiện theo thứ tự, dùng RETURN để thoát sớm khi thất bại. INSERT nếu tất cả đều thỏa.

```
CREATE OR REPLACE PROCEDURE enroll_student
(p_studentid IN NUMBER,
 p_classid   IN NUMBER)
IS
  v_check    NUMBER;
  v_capacity  NUMBER;
  v_enrolled NUMBER;
BEGIN
  -- DK1: Sinh vien phai ton tai
  SELECT COUNT(*) INTO v_check FROM student WHERE studentid =
p_studentid;
  IF v_check = 0 THEN
    DBMS_OUTPUT.PUT_LINE('[LOI] Sinh vien ' || p_studentid || ' khong
ton tai!');
    RETURN;
  END IF;

  -- DK2: Lop hoc phai ton tai
  SELECT COUNT(*) INTO v_check FROM class WHERE classid =
p_classid;
  IF v_check = 0 THEN
    DBMS_OUTPUT.PUT_LINE('[LOI] Lop hoc ' || p_classid || ' khong ton
tai!');
    RETURN;
  END IF;
```

```
-- DK3: Kiem tra con cho trong

SELECT capacity INTO v_capacity FROM class WHERE classid =
p_classid;

SELECT COUNT(*) INTO v_enrolled FROM enrollment WHERE classid =
p_classid;

IF v_enrolled >= v_capacity THEN

    DBMS_OUTPUT.PUT_LINE('[LOI] Lop ' || p_classid || ' da day! ('
        || v_enrolled || '/' || v_capacity || ');

    RETURN;

END IF;


-- DK4: Sinh vien chua dang ky lop nay
SELECT COUNT(*) INTO v_check FROM enrollment
WHERE studentid = p_studentid AND classid = p_classid;
IF v_check > 0 THEN

    DBMS_OUTPUT.PUT_LINE('[LOI] Sinh vien da dang ky lop nay roi!');

    RETURN;

END IF;


-- DK5: Sinh vien chua qua 3 lop
SELECT COUNT(*) INTO v_check FROM enrollment WHERE studentid =
p_studentid;
IF v_check >= 3 THEN

    DBMS_OUTPUT.PUT_LINE('[LOI] Sinh vien da dang ky du 3 lop!');

    RETURN;

END IF;


-- Tat ca OK: INSERT
INSERT INTO enrollment
(studentid, classid, enrolldate, createdby, createddate,
modifiedby, modifieddate)
```

```
VALUES
    (p_studentid, p_classid, SYSDATE, USER, SYSDATE, USER,
    SYSDATE);
COMMIT;
DBMS_OUTPUT.PUT_LINE('[OK] Dang ky thanh cong! SV ' || p_studentid
    || ' -> Lop ' || p_classid);
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('[LOI HE THONG] ' || SQLERRM);
END enroll_student;
/

-- Kiem tra:
BEGIN
    enroll_student(101, 5); -- Hop le
    enroll_student(999, 5); -- SV khong ton tai
    enroll_student(101, 999); -- Lop khong ton tai
END;
/
```

Câu 2.2: Thủ tục update_final_grade - cập nhật điểm tổng kết

Phân tích: Lưu điểm cũ trước khi UPDATE. Cập nhật cả bảng ENROLLMENT lẫn GRADE bằng MERGE INTO.

```
CREATE OR REPLACE PROCEDURE update_final_grade
(p_studentid IN NUMBER,
 p_classid   IN NUMBER,
 p_grade     IN NUMBER)
IS
  v_check  NUMBER;
  v_old_grade NUMBER;
BEGIN
  -- Kiểm tra điểm hợp lệ
  IF p_grade < 0 OR p_grade > 100 THEN
    DBMS_OUTPUT.PUT_LINE('[LOI] Điểm không hợp lệ! Phải từ 0 đến 100.');
```

```
    RETURN;
  END IF;

  -- Kiểm tra cap (StudentID, ClassID) tồn tại trong ENROLLMENT
  SELECT COUNT(*) INTO v_check FROM enrollment
  WHERE studentid = p_studentid AND classid = p_classid;
  IF v_check = 0 THEN
    DBMS_OUTPUT.PUT_LINE('[LOI] Sinh viên chưa đang ký lớp này!');
```

```
    RETURN;
  END IF;

  -- Lưu điểm cũ
  SELECT finalgrade INTO v_old_grade FROM enrollment
  WHERE studentid = p_studentid AND classid = p_classid;
```



```
-- Cap nhat ENROLLMENT
UPDATE enrollment
SET   finalgrade = p_grade,
      modifiedby = USER, modifieddate = SYSDATE
WHERE studentid = p_studentid AND classid = p_classid;

-- Dong bo sang bang GRADE bang MERGE INTO
MERGE INTO grade g
USING (SELECT p_studentid AS sid, p_classid AS cid FROM DUAL) src
ON (g.studentid = src.sid AND g.classid = src.cid)
WHEN MATCHED THEN
    UPDATE SET g.grade = p_grade,
              g.modifiedby = USER, g.modifieddate = SYSDATE
WHEN NOT MATCHED THEN
    INSERT (studentid, classid, grade, createdby, createddate,
            modifiedby, modifieddate)
    VALUES (p_studentid, p_classid, p_grade,
            USER, SYSDATE, USER, SYSDATE);

COMMIT;

DBMS_OUTPUT.PUT_LINE('[OK] Da cap nhat diem SV ' || p_studentid
                    || ' lop ' || p_classid
                    || ': Cu=' || NVL(TO_CHAR(v_old_grade), 'NULL')
                    || ' -> Moi=' || p_grade);

EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('[LOI] ' || SQLERRM);
END update_final_grade;
```

Câu 2.3: Thủ tục transfer_student - chuyển lớp cho sinh viên

Phân tích: Dùng SAVEPOINT để có thể ROLLBACK từng bước. Tuần tự: kiểm tra → SAVEPOINT → DELETE → SAVEPOINT → INSERT.

```
CREATE OR REPLACE PROCEDURE transfer_student
(p_studentid IN NUMBER,
 p_old_classid IN NUMBER,
 p_new_classid IN NUMBER)
IS
  v_check NUMBER;
  v_capacity NUMBER;
  v_enrolled NUMBER;
BEGIN
  -- DK1: Sinh viên đang học ở lớp cũ
  SELECT COUNT(*) INTO v_check FROM enrollment
  WHERE studentid = p_studentid AND classid = p_old_classid;
  IF v_check = 0 THEN
    DBMS_OUTPUT.PUT_LINE('[LOI] Sinh viên không đang học lớp ' ||
    p_old_classid);
    RETURN;
  END IF;

  -- DK2: Lớp mới còn chỗ trống
  SELECT capacity INTO v_capacity FROM class WHERE classid =
  p_new_classid;
  SELECT COUNT(*) INTO v_enrolled FROM enrollment WHERE classid =
  p_new_classid;
  IF v_enrolled >= v_capacity THEN
    DBMS_OUTPUT.PUT_LINE('[LOI] Lớp mới ' || p_new_classid || ' đã
    đầy!');
    RETURN;
  END IF;
```

```
-- DK3: Sinh vien chua dang ky lop moi
SELECT COUNT(*) INTO v_check FROM enrollment
WHERE studentid = p_studentid AND classid = p_new_classid;
IF v_check > 0 THEN
    DBMS_OUTPUT.PUT_LINE('[LOI] Sinh vien da o trong lop moi roi!');
    RETURN;
END IF;

-- Tat ca OK: thuc hien chuyen lop
SAVEPOINT sp_truoc_chuyen;

-- Buoc 1: Xoa khoi lop cu
DELETE FROM enrollment
WHERE studentid = p_studentid AND classid = p_old_classid;
SAVEPOINT sp_sau_xoa;

-- Buoc 2: Them vao lop moi
INSERT INTO enrollment
(studentid, classid, enrolldate, createdby, createddate,
modifiedby, modifieddate)
VALUES
(p_studentid, p_new_classid, SYSDATE, USER, SYSDATE, USER,
SYSDATE);

COMMIT;
DBMS_OUTPUT.PUT_LINE('[OK] Da chuyen SV ' || p_studentid
|| ' tu lop ' || p_old_classid
|| ' sang lop ' || p_new_classid);
EXCEPTION
```

```

WHEN OTHERS THEN
    ROLLBACK TO sp_truoc_chuyen;
    DBMS_OUTPUT.PUT_LINE('[LOI] Chuyen lop that bai: ' || SQLERRM);
    DBMS_OUTPUT.PUT_LINE('Da rollback ve trang thai ban dau.');
```

END transfer_student;

/

Câu 2.4: Thủ tục report_class_detail — in báo cáo chi tiết lớp học

Phân tích: Dùng cursor FOR LOOP duyệt danh sách SV. Biến đếm STT. CASE WHEN cho xếp loại. LPAD/RPAD căn chỉnh output.

```

CREATE OR REPLACE PROCEDURE report_class_detail
(p_classid IN NUMBER)
IS
    v_check  NUMBER;
    v_course VARCHAR2(50);
    v_courseno NUMBER;
    v_gv     VARCHAR2(50);
    v_loc    VARCHAR2(50);
    v_cap    NUMBER;
    v_stt    NUMBER := 0;
    v_tong   NUMBER := 0;
    v_sum_d  NUMBER := 0;
    v_co_d   NUMBER := 0;
    v_grade_txt VARCHAR2(15);
BEGIN
    -- Kiểm tra lớp tồn tại
    SELECT COUNT(*) INTO v_check FROM class WHERE classid =
    p_classid;
    IF v_check = 0 THEN
        DBMS_OUTPUT.PUT_LINE('Lop hoc ' || p_classid || ' không tồn tại!');
```

```
        RETURN;
    END IF;

    -- Lay thông tin lop
    SELECT co.description, co.courseno,
           i.firstname || ' ' || i.lastname,
           cl.location, cl.capacity
    INTO   v_course, v_courseno, v_gv, v_loc, v_cap
    FROM   class cl
           JOIN course co   ON cl.courseno = co.courseno
           JOIN instructor i ON cl.instructorid = i.instructorid
    WHERE  cl.classid = p_classid;

    -- In header bao cao
    DBMS_OUTPUT.PUT_LINE('=== BAO CAO LOP HOC: ' || p_classid || '
===');
    DBMS_OUTPUT.PUT_LINE('Mon hoc : ' || v_courseno || ' - ' || v_course);
    DBMS_OUTPUT.PUT_LINE('Giao vien: ' || v_gv);
    DBMS_OUTPUT.PUT_LINE('Phong hoc: ' || NVL(v_loc, 'Chua xep
phong'));
    DBMS_OUTPUT.PUT_LINE('Suc chua : ' || v_cap || ' cho');
    DBMS_OUTPUT.PUT_LINE(RPAD('-',50,'-'));
    DBMS_OUTPUT.PUT_LINE('DANH SACH SINH VIEN:');
    DBMS_OUTPUT.PUT_LINE(RPAD('STT',4) || ' | ' || RPAD('Ho Ten',20)
                           || ' | ' || LPAD('Diem TK',8) || ' | Xep loai');
    DBMS_OUTPUT.PUT_LINE(RPAD('-',50,'-'));

    -- Duyệt danh sách sinh viên
    FOR rec IN (
        SELECT s.firstname || ' ' || s.lastname AS ho_ten,
               e.finalgrade
```

```
FROM enrollment e
      JOIN student s ON e.studentid = s.studentid
WHERE e.classid = p_classid
ORDER BY s.lastname, s.firstname
) LOOP
    v_stt := v_stt + 1;
    v_tong := v_tong + 1;

    -- Xep loai
    IF rec.finalgrade IS NULL THEN
        v_grade_txt := 'Chua co diem';
    ELSIF rec.finalgrade >= 90 THEN
        v_grade_txt := 'A';
        v_sum_d := v_sum_d + rec.finalgrade; v_co_d := v_co_d + 1;
    ELSIF rec.finalgrade >= 80 THEN
        v_grade_txt := 'B';
        v_sum_d := v_sum_d + rec.finalgrade; v_co_d := v_co_d + 1;
    ELSIF rec.finalgrade >= 70 THEN
        v_grade_txt := 'C';
        v_sum_d := v_sum_d + rec.finalgrade; v_co_d := v_co_d + 1;
    ELSIF rec.finalgrade >= 50 THEN
        v_grade_txt := 'D';
        v_sum_d := v_sum_d + rec.finalgrade; v_co_d := v_co_d + 1;
    ELSE
        v_grade_txt := 'F';
        v_sum_d := v_sum_d + rec.finalgrade; v_co_d := v_co_d + 1;
    END IF;

    DBMS_OUTPUT.PUT_LINE(
        LPAD(v_stt, 3) || ' | '
```

```
        || RPAD(rec.ho_ten, 20) || ' | '
        || LPAD(NVL(TO_CHAR(rec.finalgrade), 'NULL'), 7) || ' | '
        || v_grade_txt
    );
END LOOP;

-- In footer bao cao
DBMS_OUTPUT.PUT_LINE(RPAD('-', 50, '-'));
DBMS_OUTPUT.PUT_LINE('Tong so sinh vien : ' || v_tong);
IF v_co_d > 0 THEN
    DBMS_OUTPUT.PUT_LINE('Diem trung binh lop: '
        || ROUND(v_sum_d / v_co_d, 2));
ELSE
    DBMS_OUTPUT.PUT_LINE('Diem trung binh lop: Chua co diem');
END IF;
END report_class_detail;
/

-- Goi thu tuc:
BEGIN
    report_class_detail(1);
END;
/
```

Câu 2.5: Thủ tục sync_grade_from_enrollment - đồng bộ điểm từ ENROLLMENT sang GRADE

Phân tích: Cursor FOR LOOP duyệt ENROLLMENT. Với mỗi hàng: kiểm tra tồn tại trong GRADE rồi INSERT hoặc UPDATE. Đếm số hàng xử lý.

```
CREATE OR REPLACE PROCEDURE sync_grade_from_enrollment
IS
    v_check    NUMBER;
    v_dem_insert NUMBER := 0;
    v_dem_update NUMBER := 0;
BEGIN
    FOR rec IN (
        SELECT studentid, classid, finalgrade
        FROM   enrollment
        WHERE  finalgrade IS NOT NULL
    ) LOOP
        -- Kiểm tra trong GRADE đã có chưa
        SELECT COUNT(*) INTO v_check FROM grade
        WHERE studentid = rec.studentid AND classid = rec.classid;

        IF v_check = 0 THEN
            -- Chưa có -> INSERT mới
            INSERT INTO grade
                (studentid, classid, grade, createdby, createddate,
                 modifiedby, modifieddate)
            VALUES
                (rec.studentid, rec.classid, rec.finalgrade,
                 USER, SYSDATE, USER, SYSDATE);
            v_dem_insert := v_dem_insert + 1;
        ELSE
            -- Đã có -> UPDATE
            UPDATE grade
```



```
        SET   grade = rec.finalgrade,
              modifiedby = USER, modifieddate = SYSDATE
        WHERE studentid = rec.studentid AND classid = rec.classid;
        v_dem_update := v_dem_update + 1;
    END IF;
END LOOP;

COMMIT;

DBMS_OUTPUT.PUT_LINE('[OK] Dong bo hoan tat!');
DBMS_OUTPUT.PUT_LINE('  So ban ghi INSERT moi : ' ||
v_dem_insert);
DBMS_OUTPUT.PUT_LINE('  So ban ghi UPDATE   : ' ||
v_dem_update);
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('[LOI] ' || SQLERRM);
END sync_grade_from_enrollment;
/

BEGIN sync_grade_from_enrollment; END; /
```

Lưu ý: Có thể thay toàn bộ vòng lặp này bằng một lệnh MERGE INTO duy nhất - đó là cách tối ưu hơn về hiệu năng khi dữ liệu lớn.

PHẦN 4: HƯỚNG DẪN BÀI 3 - TRIGGER**Câu 3.1:** Trigger trg_check_capacity - kiểm tra sức chứa khi đăng ký

Phân tích: BEFORE INSERT để có thể RAISE_APPLICATION_ERROR chặn INSERT. Đọc COUNT và Capacity, so sánh và từ chối nếu đầy.

```
CREATE OR REPLACE TRIGGER trg_check_capacity
BEFORE INSERT ON enrollment
FOR EACH ROW
DECLARE
    v_capacity NUMBER;
    v_enrolled NUMBER;
BEGIN
    -- Lay suc chua lop hoc
    SELECT capacity INTO v_capacity
    FROM   class WHERE classid = :NEW.classid;

    -- Dem so SV hien da dang ky
    SELECT COUNT(*) INTO v_enrolled
    FROM   enrollment WHERE classid = :NEW.classid;

    -- Tu choi neu lop da day
    IF v_enrolled >= v_capacity THEN
        RAISE_APPLICATION_ERROR(
            -20010,
            'LOI: Lop ' || :NEW.classid || ' da day! ('
            || v_enrolled || '/' || v_capacity || ' cho)'
        );
    END IF;
END trg_check_capacity;
/
```

```
-- Kiểm tra trigger (dang ky vao lop da day):
```

```
INSERT INTO enrollment
```

```
(studentid, classid, enrolldate, createdby, createddate,  
modifiedby, modifieddate)
```

```
VALUES (999, 1, SYSDATE, USER, SYSDATE, USER, SYSDATE);
```

Giải thích: Trigger BEFORE INSERT đọc COUNT() từ bảng ENROLLMENT - bảng chưa bị thay đổi tại thời điểm trigger chạy (vì là BEFORE), nên không gây lỗi ORA-04091 mutating table.*

Câu 3.2: Trigger trg_grade_audit_log - ghi nhật ký thay đổi điểm

Phân tích: AFTER UPDATE OF FinalGrade để :OLD và :NEW đều có giá trị chính xác. Kiểm tra thực sự thay đổi trước khi ghi log.

```
CREATE TABLE grade_audit_log (
```

```
log_id    NUMBER GENERATED ALWAYS AS IDENTITY,
```

```
studentid NUMBER(8),
```

```
classid   NUMBER(8),
```

```
grade_cu  NUMBER(3),
```

```
grade_moi NUMBER(3),
```

```
nguoi_sua VARCHAR2(30),
```

```
thoi_gian DATE
```

```
);
```

```
/
```

```
CREATE OR REPLACE TRIGGER trg_grade_audit_log
```

```
AFTER UPDATE OF finalgrade ON enrollment
```

```
FOR EACH ROW
```

```
BEGIN
```

```
-- Chỉ ghi log khi điểm THỰC SỰ thay đổi
```

```
IF (:OLD.finalgrade IS NULL AND :NEW.finalgrade IS NOT NULL)
```

```
OR (:OLD.finalgrade IS NOT NULL AND :NEW.finalgrade IS NULL)
```

```

        OR (:OLD.finalgrade != :NEW.finalgrade)
    THEN
        INSERT INTO grade_audit_log
            (studentid, classid, grade_cu, grade_moi, nguoi_sua, thoi_gian)
        VALUES
            (:OLD.studentid, :OLD.classid, :OLD.finalgrade,
            :NEW.finalgrade, USER, SYSDATE);
    END IF;
END trg_grade_audit_log;
/

-- Kiểm tra trigger:
UPDATE enrollment SET finalgrade = 85
WHERE studentid = 101 AND classid = 1;
COMMIT;

SELECT * FROM grade_audit_log;

```

Lưu ý: Không thể dùng :OLD.col != :NEW.col trực tiếp khi một trong hai có thể là NULL (NULL != NULL trả về NULL chứ không phải TRUE). Cần kiểm tra cả 3 trường hợp như code trên.

Câu 3.3: Trigger trg_prevent_course_delete - ngăn xóa môn học đang có lớp

Phân tích: BEFORE DELETE: kiểm tra số lớp của môn đó trong CLASS. Nếu có lớp thì RAISE_APPLICATION_ERROR. Nếu không có thì tự động tiếp tục xóa.

```

CREATE OR REPLACE TRIGGER trg_prevent_course_delete
    BEFORE DELETE ON course
    FOR EACH ROW
    DECLARE
        v_so_lop NUMBER;
    BEGIN

```

```

SELECT COUNT(*) INTO v_so_lop
FROM   class WHERE courseno = :OLD.courseno;

IF v_so_lop > 0 THEN
    RAISE_APPLICATION_ERROR(
        -20020,
        'Khong the xoa mon hoc ' || :OLD.courseno ||
        ' (' || :OLD.description || ') ' ||
        'vi con ' || v_so_lop || ' lop hoc dang ton tai!'
    );
END IF;

-- Neu v_so_lop = 0: trigger ket thuc binh thuong, Oracle tien hanh xoa
END trg_prevent_course_delete;
/

-- Kiem tra: xoa mon co lop (se bao loi)
DELETE FROM course WHERE courseno = 10;

-- Kiem tra: xoa mon khong co lop (thanh cong)
DELETE FROM course WHERE courseno = 999; -- Mon khong co lop nao
ROLLBACK;

```

Câu 3.4: Trigger trg_update_grade_summary - cập nhật bảng thống kê tự động

Phân tích: AFTER INSERT OR UPDATE OR DELETE: mỗi sự kiện dùng CASE WHEN INSERTING/UPDATING/DELETING để lấy đúng ClassID. Sau đó MERGE INTO cập nhật class_grade_summary.

```

CREATE TABLE class_grade_summary (
    classid    NUMBER(8) PRIMARY KEY,
    so_sv      NUMBER,
    diem_tb    NUMBER(5,2),

```

```
diem_cao_nhat NUMBER(3),
diem_thap_nhat NUMBER(3),
cap_nhat_luc DATE
);
/

CREATE OR REPLACE TRIGGER trg_update_grade_summary
AFTER INSERT OR UPDATE OR DELETE ON enrollment
FOR EACH ROW
DECLARE
v_classid NUMBER;
v_so_sv NUMBER;
v_diem_tb NUMBER;
v_max_d NUMBER;
v_min_d NUMBER;
BEGIN
-- Lay ClassID dua tren loi su kien
IF INSERTING OR UPDATING THEN
v_classid := :NEW.classid;
ELSE -- DELETING
v_classid := :OLD.classid;
END IF;

-- Tinh lai thong ke cho lop bi anh huong
SELECT COUNT(finalgrade),
ROUND(AVG(finalgrade), 2),
MAX(finalgrade),
MIN(finalgrade)
INTO v_so_sv, v_diem_tb, v_max_d, v_min_d
FROM enrollment
```

```
WHERE classid = v_classid AND finalgrade IS NOT NULL;

-- MERGE INTO cap nhat hoac them moi
MERGE INTO class_grade_summary cgs
USING (SELECT v_classid AS cid FROM DUAL) src
ON (cgs.classid = src.cid)
WHEN MATCHED THEN
    UPDATE SET
        so_sv      = v_so_sv,
        diem_tb     = v_diem_tb,
        diem_cao_nhat = v_max_d,
        diem_thap_nhat = v_min_d,
        cap_nhat_luc = SYSDATE
WHEN NOT MATCHED THEN
    INSERT (classid, so_sv, diem_tb, diem_cao_nhat, diem_thap_nhat,
        cap_nhat_luc)
        VALUES (v_classid, v_so_sv, v_diem_tb, v_max_d, v_min_d,
        SYSDATE);
END trg_update_grade_summary;
/

-- Kiem tra trigger:
UPDATE enrollment SET finalgrade = 90 WHERE studentid=101 AND
classid=1;
COMMIT;
SELECT * FROM class_grade_summary WHERE classid = 1;
```

Giải thích: Trigger AFTER phù hợp ở đây vì ta chỉ đọc ENROLLMENT (để tính thống kê) chứ không ghi vào chính ENROLLMENT - nên không gây lỗi mutating table. Dữ liệu ghi là bảng class_grade_summary (khác bảng kích hoạt trigger).

PHẦN 5: HƯỚNG DẪN BÀI 4 - TỔNG HỢP**Câu 4.1:** Hệ thống báo cáo hoàn chỉnh - View + Procedure + Cursor

Phân tích: Bước 1: Tạo view vw_instructor_workload. Bước 2: Thủ tục print_system_report dùng SELECT INTO cho tổng thể, sau đó cursor FOR LOOP duyệt qua các view.

Bước 1: Tạo view vw_instructor_workload:

```
CREATE OR REPLACE VIEW vw_instructor_workload AS
SELECT i.instructorid,
       i.firstname || ' ' || i.lastname      AS ho_ten,
       COUNT(DISTINCT cl.classid)           AS so_lop,
       COUNT(e.studentid)                   AS tong_sv,
       ROUND(AVG(e.finalgrade), 2)          AS diem_tb_chung,
       CASE
         WHEN COUNT(DISTINCT cl.classid) >= 3 THEN 'Ban nhieu'
         WHEN COUNT(DISTINCT cl.classid) = 2 THEN 'Binh thuong'
         ELSE 'Nhe nhang'
       END                                  AS muc_ban
FROM   instructor i
       LEFT JOIN class cl   ON i.instructorid = cl.instructorid
       LEFT JOIN enrollment e ON cl.classid = e.classid
GROUP BY i.instructorid, i.firstname, i.lastname
ORDER BY so_lop DESC;

SELECT * FROM vw_instructor_workload;
```

Bước 2: Thủ tục print_system_report:

```
CREATE OR REPLACE PROCEDURE print_system_report
IS
  v_so_mon NUMBER;
  v_so_lop NUMBER;
```



```

v_so_sv NUMBER;
v_so_gv NUMBER;
BEGIN
-- Lay so lieu tong the
SELECT COUNT(*) INTO v_so_mon FROM course;
SELECT COUNT(*) INTO v_so_lop FROM class;
SELECT COUNT(*) INTO v_so_sv FROM student;
SELECT COUNT(*) INTO v_so_gv FROM instructor;

-- In header

DBMS_OUTPUT.PUT_LINE('=====');

DBMS_OUTPUT.PUT_LINE(' BAO CAO TOAN HE THONG QUAN LY KHOA HOC');

DBMS_OUTPUT.PUT_LINE('=====');

DBMS_OUTPUT.PUT_LINE('Tong so mon hoc : ' || v_so_mon);
DBMS_OUTPUT.PUT_LINE('Tong so lop hoc : ' || v_so_lop);
DBMS_OUTPUT.PUT_LINE('Tong so sinh vien: ' || v_so_sv);
DBMS_OUTPUT.PUT_LINE('Tong so giao vien: ' || v_so_gv);
DBMS_OUTPUT.PUT_LINE(RPAD('-',50,'-'));

-- Phan 1: Thong ke giao vien (dung view vw_instructor_workload)
DBMS_OUTPUT.PUT_LINE('THONG KE GIAO VIEN:');
FOR rec IN (SELECT * FROM vw_instructor_workload) LOOP
DBMS_OUTPUT.PUT_LINE(
' ' || RPAD(rec.ho_ten, 25)
|| ' | ' || LPAD(rec.so_lop, 2) || ' lop'
|| ' | ' || LPAD(rec.tong_sv, 3) || ' SV'
|| ' | DTB: ' || NVL(TO_CHAR(rec.diem_tb_chung),'--')

```

```
        || ' ' || rec.muc_ban
    );
END LOOP;
DBMS_OUTPUT.PUT_LINE(RPAD('-',50,'-'));

-- Phan 2: Top 3 mon hoc (dung view vw_top_courses)
DBMS_OUTPUT.PUT_LINE('TOP 3 MON HOC DUOC DANG KY
NHIEU NHAT:');
FOR rec IN (SELECT * FROM vw_top_courses WHERE hang <= 3) LOOP
    DBMS_OUTPUT.PUT_LINE(
        ' ' || rec.hang || ' '
        || RPAD(rec.description, 30)
        || ' - ' || rec.tong_dk || ' luot dang ky'
    );
END LOOP;

DBMS_OUTPUT.PUT_LINE('=====
=====');
END print_system_report;
/

-- Chay bao cao:
SET SERVEROUTPUT ON SIZE 1000000;
BEGIN print_system_report; END;
/
```

PHẦN 6: BẢNG LỖI THƯỜNG GẶP VÀ CHECKLIST

6.1. Lỗi Phổ Biến Trong Buổi 4

Lỗi	Nguyên nhân	Cách khắc phục
ORA-42399: read-only view	Thực hiện DML trên view WITH READ ONLY	Thao tác trực tiếp trên bảng cơ sở
ORA-01402: WITH CHECK OPTION violation	INSERT/UPDATE qua view vi phạm điều kiện WHERE	Đảm bảo dữ liệu thỏa mãn điều kiện WHERE của view
ORA-04091: table is mutating	Trigger AFTER đọc/ghi chính bảng đang bị thay đổi	Đổi thành BEFORE, hoặc dùng Compound Trigger
ORA-20xxx: user-defined error	RAISE_APPLICATION_ERROR trong trigger/procedure	Đọc thông báo lỗi; kiểm tra dữ liệu vi phạm
ORA-04098: trigger invalid	Trigger compile lỗi nhưng vẫn được tạo	SHOW ERRORS TRIGGER ten_trigger; sửa và biên dịch lại
View bị INVALID	Bảng cơ sở của view bị thay đổi cấu trúc	CREATE OR REPLACE VIEW để biên dịch lại
RANK() lỗi trong WHERE	Hàm phân tích không được dùng trực tiếp trong WHERE	Bọc trong Inline View (subquery trong FROM)

6.2. Checklist Kiểm Tra Trước Khi Nộp Bài

- Đã chạy SET SERVEROUTPUT ON; trước mỗi phiên làm việc
- Mỗi view: chạy SELECT * FROM ten_view; để xác nhận hiển thị đúng
- Mỗi procedure: gọi thử với dữ liệu hợp lệ VÀ dữ liệu không hợp lệ
- Mỗi trigger: thử cả trường hợp cho phép lẫn trường hợp bị từ chối
- Kiểm tra trạng thái đối tượng: **SELECT object_name, object_type, status FROM user_objects WHERE status = 'INVALID';**
- Xem lỗi compile: **SHOW ERRORS;** sau mỗi lệnh CREATE
- Đã COMMIT sau mỗi thao tác DML quan trọng
- Lưu tất cả vào file Tuan4_MSSV.sql

Good Luck!